Claude Code for Scientific Research

A cheat sheet for using an AI coding agent to speed up analysis, scripting, and data wrangling in the lab.

How It Works

- **Not just a chatbot:** An agentic assistant that runs in a terminal (or IDE) inside your project folder — it can read your files, write/edit code, and run programs, not just chat.
- **The basic loop:** You give it a task → it reads relevant files → proposes or makes changes → runs them → shows you the result → you review and iterate.
- **Context window:** It "remembers" your conversation and any files it has read during the session (the context window). Very long sessions get automatically summarized.
- **You approve the risky stuff:** By default it asks permission before risky actions (editing files, running commands, deleting things). You stay in control.
- **Plan Mode:** For big or ambiguous tasks, it first researches and drafts a plan — no files touched yet. You can revise the plan, then must explicitly approve before it executes. See "Plan Mode in Detail" below.

Using the GUI App (No Terminal Needed)

- **1.** Open the Claude Code app (desktop app or IDE extension) instead of a terminal window.
- **2.** Pick your project folder — this is the folder Claude can see and work in.
- **3.** Type your task in plain English in the chat pane, same as you would in the terminal.
- **4.** Review proposed edits inline — changes are shown as highlighted diffs; click to approve or reject each one.

The CLI (terminal) and the GUI app are the same underlying tool — just a different front end. Everything in this cheat sheet applies to both.

Plan Mode in Detail

- **What triggers it:** Ambiguous or high-stakes requests (a big refactor, a new analysis approach, a grant draft) — or you can explicitly ask it to plan first.
- **What happens:** Claude explores your files/data and writes up an approach — no edits happen during this stage.
- **You can revise & annotate it:** Read the draft, ask for changes, question assumptions, or comment on specific parts — it will redraft before you approve.
- **Nothing runs until you approve:** Nothing changes on disk until you explicitly approve the plan. This makes it a cheap way to catch a wrong approach before spending tokens executing it.

What To Use It For (Research & Writing Examples)

Task	Example prompt
Analysis scripts	*"Write a Python script that reads all plate-reader CSVs in data/, subtracts blanks, and outputs one summary table."*
Data cleaning	*"These three experiment spreadsheets have inconsistent column names — write a script to standardize and merge them."*
Plotting / QC	*"Plot a dose-response curve from results.csv with error bars, and flag any wells that look like outliers."*
Flow cytometry / imaging	*"Here's my flow cytometry export — write a gating script on these markers and output a summary table."*
Sequencing (bulk/scRNA-seq)	*"Load these count matrices, run QC filtering and normalization, and cluster the cells — write the pipeline as a reusable script."*
Spatial multiomics	*"I have Visium spatial transcriptomics counts plus the matched H&E image — cluster spots, annotate cell types, and overlay results on the tissue image."*
Pipeline glue	*"Rename and organize these 200 sequencing files by sample ID using the naming key in samples.xlsx."*

Task	Example prompt
Explaining output	*"Explain in plain language what this statsmodels regression summary is telling me."*
Paper review / critique	*"Critique this manuscript's methods section — are the statistical tests appropriate for this study design?"*
Explaining papers & formulae	*"Walk me through equation 3 in this paper in plain language, and show the derivation step by step."*
Grants & fellowships	*"Help me tighten the specific-aims page against this funder's page limit without losing the key logic."*

Not a substitute for domain judgment or statistical consulting — great for accelerating implementation, not for validating scientific conclusions.

Checking Your Usage

- **Where to look:** Claude Code: run the /usage command. • claude.ai (web): profile icon → Settings → Usage.
- **Context window:** How much of the current conversation + files fit in this one session. Not a time-based limit — it runs out mid-conversation, not mid-week.
- **5-hour limit:** A rolling budget shared across Claude.ai, Claude Code, and the desktop app. Resets 5 hours after your first message in that window.
- **Weekly limit:** A separate fixed cap that resets on your assigned day each week. Applies even if your 5-hour budget still has room left.
- **Per-model weekly quotas:** The highest-capability model (Fable) gets its own additional weekly quota on top of the shared "all models" quota, since it costs the most to run.

Example panel (illustrative): Context window 203.9k / 967k (21%) · 5-hour limit 0%, resets in 4h 53m · Weekly · all models 7%, resets Thu 1:00 AM · Weekly · Fable 0%.

Model Tiers & Effort

Model	Capability	Speed	Typical use
Fable	Highest	Slower	Long-running agents, deep multi-step research
Opus	Very high	Moderate	Complex agentic coding, large refactors
Sonnet	High (default)	Fast	Everyday coding, analysis, writing — best default
Haiku	Good, near-frontier	Fastest	High-volume, quick tasks, sub-agent helpers

Context windows: Fable/Opus/Sonnet ≈ 1M tokens; Haiku ≈ 200k tokens. Higher-capability models draw down usage limits faster per token.

Effort	What it controls
Low	*Fastest, most token-efficient — simple/quick tasks, high-volume work.*
Medium	*Balanced cost vs. quality — solid default for routine work.*
High	*Full reasoning depth — default for most models on complex tasks.*
X-High / Max	*Maximum reasoning for long, hard problems — highest token cost.*

Best Practices for Optimizing Token Usage

- **Be specific & scoped:** Ask for narrow, well-scoped tasks ("clean column names in results.csv") rather than "look at my whole project and fix things."
- **Point to exact files:** If you know which file/folder is relevant, say so — it avoids an expensive, broad search.
- **Don't paste large data:** Point to the file path and let it read only what's needed, rather than pasting huge CSVs or logs into the chat.
- **New session per task:** Start a new conversation per unrelated task — don't let one session's history balloon with irrelevant context.

- **Prefer scripts over chat-only work:** Have it write a reusable script instead of doing one-off manual data massaging in chat — scripts are cheap to keep, chat history is not.
- **Plan before large changes:** For big changes, use Plan Mode (or explicitly ask for a plan) before it touches anything — cheaper to redirect a plan than to redo a large wrong change.
- **Checkpoint on long tasks:** On long tasks, check in periodically rather than letting it run unsupervised for a very long stretch.

Quick Start — Try It Yourself

Same tool either way — pick whichever fits how you already work.

Terminal (CLI)	GUI App
1. Open a terminal and navigate into your project folder (e.g., cd Documents/my-experiment).	1. Open the Claude Code app (desktop app or IDE extension) — no terminal needed.
2. Run the claude command to start a session.	2. Pick your project folder — this is what Claude can see and work in.
3. Describe your task in plain English — mention the specific file(s) if you know them.	3. Type your task in plain English in the chat pane.
4. Review what it proposes in the terminal, approve or adjust, and iterate.	4. Review proposed edits inline — highlighted diffs; click to approve or reject.

Glossary

Token	A chunk of text (roughly ¾ of a word) — the basic unit Claude reads and writes; usage and cost scale with tokens.
Context window	The amount of conversation + file content Claude can "hold in mind" at once during a session.
Session	One continuous conversation with Claude Code; starting a new one clears prior context.
Agent	An AI system that can take actions (read/write files, run commands) toward a goal, not just answer questions.
Prompt	The instruction/request you give Claude.
Plan Mode	A mode where Claude drafts an approach before making any changes; you can revise it and must approve before it executes.
5-hour limit	A rolling usage budget, shared across Claude.ai/Code/desktop, that resets 5 hours after your first message in the window.
Weekly limit	A fixed weekly usage cap that resets on your assigned day; applies even with 5-hour budget remaining.
Effort	A setting (low/medium/high/x-high/max) controlling how much reasoning depth — and how many tokens — a response uses.